

String Programms

Anagram

```
package StringPrograms;

import java.util.HashMap;
import java.util.Scanner;

public class Anagram {

    public static boolean areAnagrams(String str1, String str2) {
        // Remove spaces and convert to lowercase
        str1 = str1.replaceAll("\\s", "").toLowerCase();
        str2 = str2.replaceAll("\\s", "").toLowerCase();

        // If lengths are different, they cannot be anagrams
        if (str1.length() != str2.length()) {
            return false;
        }

        // Create a HashMap to store character frequencies
        HashMap<Character, Integer> charCountMap = new HashMap<>();

        // Count frequency of characters in str1
        for (char ch1 : str1.toCharArray()) {
            charCountMap.put(ch1, charCountMap.getOrDefault(ch1, 0) + 1);
        }

        // Decrease frequency based on str2
        for (char ch2 : str2.toCharArray()) {
            if (charCountMap.containsKey(ch2)) {
                charCountMap.put(ch2, charCountMap.get(ch2) - 1);
            }
            else{
                return false;
            }
        }

        // Check if all character frequencies are zero
        for (int count : charCountMap.values()) {
            if (count != 0) {
                return false;
            }
        }
        // return true when all characters in both strings are same
        return true;
    }
}
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Taking input from the user
    System.out.print("Enter first string: ");
    String str1 = sc.nextLine();

    System.out.print("Enter second string: ");
    String str2 = sc.nextLine();

    // Check if they are anagrams
    if (areAnagrams(str1, str2)) {
        System.out.println("The strings are anagrams.");
    } else {
        System.out.println("The strings are NOT anagrams.");
    }

    sc.close();
}
```

CheckPalindrome

```
package StringPrograms;

public class CheckPalindrome {

    public static boolean isPalindome(char arr[]){

        int s = 0;
        int e = arr.length - 1;

        while(s<e){

            if(arr[s] != arr[e]){
                return false;
            }
            // if equal then moves pointer to check Another charchter
            s++;
            e--;
        }
        return true;
    }

    public static void main(String[] args) {

        String str = "NooN";
        char a[] = str.toCharArray();
```

```
        if(isPalindome(a)){
            System.out.println("The Given String is palindrome");
        }
        else{
            System.out.println("The Given String is not palindrome");
        }
    }
}
```

CheckPalindrome2

```
package StringPrograms;

public class CheckPalindrome2 {

    public static boolean isPalindome(String st){

        int s = 0;
        int e = st.length() - 1;

        while(s<=e){
            if(st.charAt(s) != st.charAt(e)){
                return false;
            }
            else{        // if equal then moves pointer to check Another charchter
                s++;
                e--;
            }
        }

        return true;
    }

    public static void main(String[] args) {

        String str = "Noon";

        if(isPalindome(str)){
            System.out.println("The Given String is palindrome");
        }
        System.out.println("The Given String is not palindrome");
    }
}
```

CheckPalindrome3

```
package StringPrograms;

public class CheckPalindrome3 {

    public static void main(String[] args) {

        String str = "Hello";
        String rev = "";

        for(int i = str.length() - 1; i >= 0; i--){

            rev = rev + str.charAt(i);    // rev = "" + o
                                         //      = o + l
        }

        System.out.println("Reverse String : " + rev);

        // to check string is palindrome or not
        if(str.equalsIgnoreCase(rev)){
            System.out.println("String is palindrome");
        }
        else{
            System.out.println("String is not a palindrome");
        }
    }
}
```

CountFrequencyOfCharachter

```
package StringPrograms;

import java.util.LinkedHashMap;
import java.util.Map;

public class CountFrequencyOfCharachter {

    public static void main(String[] args) {
        String str = "hello world";

        // Create a HashMap to store character frequencies
        Map<Character, Integer> fm = new LinkedHashMap<>();

        // Loop through each character in the string
        for (char ch : str.toCharArray()) {
            if (ch != ' ') { // Ignore spaces
                fm.put(ch, fm.getOrDefault(ch, 0) + 1);
            }
        }
    }
}
```

```

        // Another way
        // for (char ch : str.toCharArray()) {
        //     if( ch != ' '){
        //         if(fm.containsKey(ch)){
        //             fm.put(ch, fm.get(ch) + 1);
        //         }
        //         else{
        //             fm.put(ch, 1);
        //         }
        //     }
        // }
        // }

        // Print the character frequencies
        for (Map.Entry<Character, Integer> entry : fm.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}

```

countHighestFreqoccurencechar

```

package StringPrograms;

// import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class countHighestFreqoccurencechar {

    public static void main(String[] args) {

        String str = "hello world";
        str = str.toLowerCase();

        // Create a HashMap to store character frequencies
        Map<Character, Integer> fm = new LinkedHashMap<>();

        // Loop through each character in the string
        for (char ch : str.toCharArray()) {
            if( ch != ' '){
                if(fm.containsKey(ch)){
                    fm.put(ch, fm.get(ch) + 1);
                }
                else{
                    fm.put(ch, 1);
                }
            }
        }

        // Printing the frquencies
        for (Map.Entry<Character, Integer> entry : fm.entrySet()) {

```

```
        System.out.println(entry.getKey() + ": " + entry.getValue());
    }

    char maxChar = '\0'; // Null character as default
    int maxFrequency = 0;

    for (Map.Entry<Character, Integer> entry : fm.entrySet()) {
        if (entry.getValue() > maxFrequency) {
            maxChar = entry.getKey();
            maxFrequency = entry.getValue();
        }
    }
    // Print the result
    System.out.println("Character with the highest occurrence: '" + maxChar +
        "' with frequency " + maxFrequency);
}
```

CountWords

```
package StringPrograms;

public class CountWords {

    public static void main(String[] args) {

        // initializing a string
        String msg = "India win the Cricket match";
        System.out.println("The given String is: " + msg);

        // Initial count of words
        int total = 1;

        // Loop to count words
        for (int i = 0; i < msg.length() - 1; i++) { // Ensure i + 1 is valid
            if ((msg.charAt(i) == ' ') && (msg.charAt(i + 1) != ' ')) {
                total++; // Increment word count
            }
        }

        // Printing the result
        System.out.println("Number of words in the given string: " + total);
    }
}
```

Deletechar

```
package StringPrograms;

public class Deletechar {

    public static void main(String[] args) {

        String src = "welcome to my youtube channel" ;
        System.out.println("String before removing character : " + src);

        int pos = 11;

        String newsrc = src.substring(0, pos) + src.substring(pos + 1);

        System.out.println("String After removing character : " + newsrc);
    }
}
```

FindDuplicatesCharacter

```
package StringPrograms;

import java.util.HashMap;
import java.util.Map;

public class FindDuplicatesCharacter {

    public static void findDuplicateCharacters(String str) {
        // Create a HashMap to store character frequencies
        Map<Character, Integer> frequencyMap = new HashMap<>();

        // Count the frequency of each character
        for (char ch : str.toCharArray()) {
            frequencyMap.put(ch, frequencyMap.getOrDefault(ch, 0) + 1);
        }

        // Print characters with a frequency greater than 1
        System.out.println("Duplicate characters in the string:");
        for (Map.Entry<Character, Integer> entry : frequencyMap.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.print(entry.getKey() + " ");
            }
        }
    }

    public static void main(String[] args) {
        String str = "programming"; // Example string
        findDuplicateCharacters(str);
    }
}
```

FindNonRepeatingChar

```
package StringPrograms;

import java.util.LinkedHashMap;
import java.util.Map;

public class FindNonRepeatingChar {

    public static void findNonRepeatingCharacters(String str) {
        Map<Character, Integer> charCount = new LinkedHashMap<>();

        // Count occurrences of each character
        for (char ch : str.toCharArray()) {
            charCount.put(ch, charCount.getOrDefault(ch, 0) + 1);
        }

        // Print non-repeating characters
        System.out.print("Non-repeating characters: ");
        for (Map.Entry<Character, Integer> entry : charCount.entrySet()) {
            if (entry.getValue() == 1) {
                System.out.print(entry.getKey() + " ");
            }
        }
    }

    public static void main(String[] args) {

        String str = "Jasmeen";
        findNonRepeatingCharacters(str);
    }
}
```

FindOccurrenceOfElement

```
package StringPrograms;

public class FindOccurrenceOfElement {

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 1, 2, 1, 4, 5, 6, 2, 3};

        // Array to store the frequency of each element
        int[] frequency = new int[arr.length];

        for (int i = 0; i < arr.length; i++) {
```



```

        if (frequency[i] == 0) { // Only count elements not yet counted

            int count = 1; // Start the count for this element

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) { // Match found
                    count++;
                    frequency[j] = -1; // Mark as counted
                }
            }
            frequency[i] = count; // Store the frequency of the element
        }
    }

    // Print the elements with their frequencies
    for (int i = 0; i < arr.length; i++) {
        if (frequency[i] > 0) { // Print only the elements with positive
frequency
            System.out.println("Element: " + arr[i] + ", Frequency: " +
frequency[i]);
        }
    }

    // Find the maximum frequency and its index
    int max = frequency[0];
    int maxIndex = 0; // To track the index of the max element

    for (int i = 1; i < frequency.length; i++) {
        if (frequency[i] > max) {
            max = frequency[i];
            maxIndex = i; // Update index of max frequency
        }
    }

    // Output the result
    System.out.println("Element with the highest frequency: Index = " +
maxIndex + ", Frequency = " + max);
    }
}

```

RemoveDuplicatesCharacterInAStringUsingSet

```

package StringPrograms;

import java.util.LinkedHashSet;

public class RemoveDuplicatesCharacterInAStringUsingSet {

    public static String RemoveDuplicates(String str){
        // Create a LinkedHashSet to store characters because it does not stores

```

```

duplicates value
    LinkedHashSet<Character> set = new LinkedHashSet<>();

    // convert to character and add it to the set (only unique value)
    for(char ch : str.toCharArray()){
        set.add(ch);          // Duplicates are automatically removed
    }

    // convert to stringBuilder
    StringBuilder res = new StringBuilder();

    // add or append unique characters to string
    for(char ch : set){
        res.append(ch);
    }

    return res.toString(); // convert to string from stringBuilder
}
public static void main(String[] args) {

    String str = "programming";
    String output = RemoveDuplicates(str);
    // System.out.println("Original: " + str);
    System.out.println("Without Duplicates: " + output);
}
}

```

RemoveDuplicatesCharacterInAStringUsingStringBuilder

```

package StringPrograms;

public class RemoveDuplicatesCharacterInAStringUsingStringBuilder {

    public static String removeDuplicateChars(String str) {
        StringBuilder sb = new StringBuilder();
        boolean[] seen = new boolean[256]; // ASCII character set

        for (char ch : str.toCharArray()) {
            if (!seen[ch]) { // If character is not seen before
                sb.append(ch); // Add to StringBuilder
                seen[ch] = true; // Mark character as seen
            }
        }
        return sb.toString(); // Convert StringBuilder to String
    }

    public static void main(String[] args) {
        String input = "programming";
    }
}

```

```
        String output = removeDuplicateChars(input);
        System.out.println("Original: " + input);
        System.out.println("Without Duplicates: " + output);
    }
}
```

RemoveNonRepeatingChar

```
package StringPrograms;

import java.util.LinkedHashMap;
import java.util.Map;

public class RemoveNonRepeatingChar{

    public static String removeNonRepeating(String str) {
        Map<Character, Integer> charCount = new LinkedHashMap<>();
        StringBuilder result = new StringBuilder();

        // Count occurrences of each character
        for (char ch : str.toCharArray()) {
            charCount.put(ch, charCount.getOrDefault(ch, 0) + 1);
        }

        // Keep only repeating characters
        for (char ch : str.toCharArray()) {
            if (charCount.get(ch) > 1) {
                result.append(ch);
            }
        }
        return result.toString();
    }

    public static void main(String[] args) {

        String str = "swiss";
        String result = removeNonRepeating(str);
        System.out.println("String after removing non-repeating characters: " +
result);
    }
}
```

RemoveSpaces

```
package StringPrograms;
```

```
public class RemoveSpaces {

    public static void main(String[] args) {

        String str = "India Won the Cricket match";
        str = str.toLowerCase();

        StringBuilder res = new StringBuilder();

        for(char ch : str.toCharArray()){
            if(ch != ' '){
                res.append(ch);
            }
        }
        System.out.println(res);
    }
}
```

RemoveVowels

```
package StringPrograms;

import java.util.Scanner;

public class RemoveVowels {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);
        System.out.print("Enter the String : ");
        String str=sc.nextLine();

        str = str.toLowerCase();

        String res="";

        for(char ch : str.toCharArray())
        {
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                continue;
            else
                res = res + ch;
        }
        System.out.println(res);

        sc.close();
    }
}
```

ReverseAString

```
package StringPrograms;
import java.util.Scanner;

public class ReverseAString {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the String");
        String input = sc.nextLine();

        StringBuilder str = new StringBuilder(input);
        String newstr = str.reverse().toString();

        System.out.println(newstr);

        sc.close();
    }
}
```

ReverseAString3

```
package StringPrograms;

public class ReverseAString3 {

    public static void main(String[] args) {

        String str = "Hello";
        String rev = "";

        for(int i = str.length() - 1; i >= 0; i--){

            rev = rev + str.charAt(i);    // rev = "" + o
                                         //      = o + l
        }

        System.out.println("Reverse String : " + rev);

        if(str.equalsIgnoreCase(rev)){
            System.out.println("String is palindrome");
        }
        else{
            System.out.println("String is not a palindrome");
        }
    }
}
```

```
}
```

```
}
```

ReverseAStringMethod2

```
package StringPrograms;

public class ReverseAStringMethod2 {

    public static void main(String[] args) {

        String str = "Noon";
        char ch[] = str.toCharArray();

        int l = 0;
        int r = ch.length - 1;

        while (l <= r) {
            // Swap characters
            char temp = ch[l];
            ch[l] = ch[r];
            ch[r] = temp;

            // Move pointers
            l++;
            r--;
        }

        String reversedStr = new String(ch);
        System.out.println("Reversed String: " + reversedStr);

        // below code is for checking palindrome or not
        // if(str.equalsIgnoreCase(reversedStr)){
        //     System.out.println("String is palindrome");
        // }
        // else{
        //     System.out.println("String is not palindrome");
        // }

    }
}
```

VowelsConsonentSpaces

```
package StringPrograms;

import java.util.Scanner;

public class VowelsConsonentSpaces {

    public static void checkCond(String str){
        int vowels = 0; int consonent = 0; int spaces = 0;

        // str = str.toLowerCase();
        for(char ch : str.toCharArray()){
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ){
                vowels++;
            }
            else if(ch == ' '){
                spaces++;
            }
            else{
                consonent++;
            }
        }

        System.out.println("Vowels : " + vowels);
        System.out.println("Consonent : " + consonent);
        System.out.println("Spaces : " + spaces);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the String : ");
        String str = sc.nextLine().toLowerCase();
        checkCond(str);

        sc.close();
    }
}
```